Auto foco (Eixo Z)

Demonstração das 4 métricas de foco

Para todas as abordagens:

- Entrada = imagem em escala de cinza I(x, y) (ou convertida);
- Saída = escalar S ≥ 0 onde maior = mais nítido.

A lógica comum é: imagem focada tem mais variação local de intensidade.

1. Variância do Laplaciano

Ideia: o operador Laplaciano é um filtro passa-alta (segunda derivada). Em regiões nítidas (bordas, texturas) ele responde forte; em regiões desfocadas, quase nada. A **variância** da resposta mede o quanto a imagem "reage" ao filtro.

Definição matemática:

$$S_{LAP} = \mathrm{Var}\left(\nabla^2 I\right) = \frac{1}{N}\sum_{x,y}\left(L(x,y) - \bar{L}\right)^2$$

onde $L = I * K_{LAP}$ e o kernel canônico é:

$$K_{LAP} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Algoritmo:

Entrada: imagem I (grayscale)

1. Se I for colorida, converter para cinza.
2. Convoluir I com o kernel Laplaciano 3×3 → L.
3. Calcular a média de L.
4. Calcular a variância de L: mean((L - mean(L))²).
5. Retornar a variância como score de foco.

Implementação:

```
import cv2, numpy as np

def metric_variance_laplacian(img):
    if img.ndim == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # CV_64F evita overflow e mantém sinal (Laplaciano tem negativos)
    L = cv2.Laplacian(img, cv2.CV_64F)
    return float(L.var())
```

2. Tenengrad

Ideia: "Tenengrad" = *Tenengrad* (Tenenbaum gradient). Em vez da segunda derivada, mede a **energia do gradiente** (primeira derivada). Usa Sobel para estimar e *Gx* e *Gy*, e soma o quadrado da magnitude. É mais robusta a ruído que o Laplaciano puro porque o Sobel já faz uma suavização embutida.

Definição matemática:

$$S_{TEN} = \frac{1}{N}\sum_{x,y}\left(G_x^2 + G_y^2\right), \quad \text{onde } G_x = I * K_x,\ G_y = I * K_y$$

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = K_x^\top$$

Algoritmo:

Entrada: imagem I (grayscale)

1. Converter para cinza, se necessário.
2. Aplicar Sobel horizontal → Gx.
3. Aplicar Sobel vertical → Gy.
4. Calcular mapa de energia: E = $Gx^2 + Gy^2$ (elemento a elemento).
5. Retornar S = média(E) [ou soma; média é mais estável entre resoluções].

Implementação:

```
def metric_tenengrad(img):
    if img.ndim == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    g = img.astype(np.float64)
    gx = cv2.Sobel(g, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(g, cv2.CV_64F, 0, 1, ksize=3)
    return float((gx * gx + gy * gy).mean())
```

Variante comum: aplicar limiar em G2 (só contar gradientes acima de T) para evitar que ruído de sensor domine em imagens muito escuras.

3. Brenner

Ideia: uma das mais antigas (Brenner, 1976). Simplesmente **quadrado da diferença entre um pixel e seu vizinho a *k* pixels de distância** (tipicamente *k*=2). Não usa convolução, é direto e muito rápido. Captura bem bordas verticais quando aplicado no eixo horizontal (e vice-versa).

Definição matemática:

$$S_{BRE} = \frac{1}{N} \sum_{x,y} \big(I(x+k, y) - I(x, y)\big)^2$$

Na prática, também se soma a versão no eixo vertical para não ficar enviesada:

$$S_{BRE} = \frac{1}{N} \sum \left[(I_{x+k} - I_x)^2 + (I_{y+k} - I_y)^2\right]$$

Algoritmo:

Entrada: imagem I (grayscale), k=2

1. Converter para cinza, se necessário;
2. Ao longo do eixo X: Dx = I[:, k:] - I[:, :-k]; Ex = mean(Dx^2);
3. Ao longo do eixo Y: Dy = I[k:, :] - I[:-k, :]; Ey = mean(Dy^2);
4. Retornar S = Ex + Ey.

Implementação:

```
def metric_brenner(img, k=2):
    if img.ndim == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    g = img.astype(np.float64)
    dx = g[:, k:] - g[:, :-k]
    dy = g[k:, :] - g[:-k, :]
    return float((dx * dx).mean() + (dy * dy).mean())
```

Na versão clássica original: apenas dx (varredura horizontal). Funciona bem quando o foco é avaliado em amostras com estrutura predominantemente vertical.

4. Energia de alta frequência via FFT

Ideia: o desfoque é um filtro **passa-baixa**: ele atenua justamente as frequências altas. No domínio de Fourier, isso é visível, a magnitude cai nas altas frequências. Se medirmos a energia do espectro fora de uma região central (baixas frequências), obtemos um indicador global de foco. É a métrica mais "física" e menos sensível a orientação de borda (não privilegia X ou Y).

Definição matemática:

$$S_{FFT} = \frac{1}{|H|}\sum_{(u,v)\in H}\left|\mathcal{F}\{I\}(u,v)\right|, \quad H = \{(u,v) : (u-u_c)^2 + (v-v_c)^2 > r^2\}$$

onde (u_c, v_c) é o centro (frequência zero) e r o raio da região de baixas frequências descartadas.

Algoritmo:

Entrada: imagem I (grayscale), raio r

1. Converter para cinza e para float;
2. Aplicar janela (Hamming/Hann) para reduzir vazamento espectral (opcional, mas recomendado);
3. Calcular F = fft2(I); deslocar o zero para o centro: fftshift;
4. Calcular magnitude: M = |F|;
5. Construir máscara H = pontos com distância ao centro > r;
6. Retornar S = média(M[H]).

Implementação:

```
def metric_fft_highfreq(img, radius_frac=1/8):
    if img.ndim == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    g = img.astype(np.float64)

    # Janela Hamming 2D para reduzir descontinuidades nas bordas
    wy = np.hamming(g.shape[0])[:, None]
    wx = np.hamming(g.shape[1])[None, :]
    g = g * wy * wx

    F = np.fft.fftshift(np.fft.fft2(g))
    M = np.abs(F)

    h, w = M.shape
    cy, cx = h // 2, w // 2
    Y, X = np.ogrid[:h, :w]
    r = int(min(h, w) * radius_frac)
    high_mask = (Y - cy) ** 2 + (X - cx) ** 2 > r * r

    return float(M[high_mask].mean())
```

Sobre o raio r: se for muito pequeno, a métrica fica dominada por baixas frequências (pouco discriminativa); se for muito grande, sobra pouco espectro (ruidosa). Um bom ponto de partida é r≈min(H,W)/8, ajustado empiricamente para o tipo de tecido da lâmina.

Comparação conceitual

Aspecto	Laplaciano	Tenengrad	Brenner	FFT alta freq.
Ordem da derivada	2ª	1ª (Sobel)	1ª (diferença)	- (espectral)
Custo computacional	baixo	médio	muito baixo	alto
Sensível a ruído	alto	médio	alto	baixo
Sensível a iluminação	alto	médio	alto	baixo
Viés direcional	não	não	sim (se só X)	não
Interpretação física	resposta passa-alta local	energia do gradiente	variação local	perda de altas freq.

Tabela comparativa

Critério	1. Variância do Laplaciano	2. Tenengrad	3. Brenner	4. Energia de alta frequência (FFT)
Fórmula	Var(∇2I)	Gx2+Gy2	(ΔkI)2	|FI|alta
Complexidade	O(N)	O(N)O(N)	O(N)	O(NlogN)
Custo relativo	baixo	médio	muito baixo	alto
Robustez a ruído	baixa	média	baixa	alta
Robustez a iluminação	baixa	média	baixa	alta
Sensibilidade a desfoque pequeno	alta	alta	média	alta
Sensibilidade a desfoque grande	média	alta	alta	média
Viés direcional	nenhum	nenhum	sim (corrigível somando X+Y)	nenhum
Precisa de imagem colorida?	não	não	não	não
Sensível a ROI vazia (fundo branco)?	sim, score cai	sim	sim	pouco (ruído distribuído)
Reprodutibilidade entre câmeras/exposições	baixa (escala com ganho)	média	baixa	alta (com normalização)
Custo de tuning	1 parâmetro (kernel)	1 parâmetro (ksize Sobel)	1 parâmetro (kk)	1 parâmetro (raio r) + janela
Uso típico na literatura	microscopia, autofoco rápido	autofoco industrial, visão	histórico, embarcado	análise de textura, microscopia avançada
Quando escolher	Quando precisar de algo simples, rápido e já validado em microscopia; melhor ponto de partida para o protótipo.	mais robustez que o Laplaciano com custo ainda baixo; ótima escolha para varredura de 40 mm.	hardware muito limitado (Arduino, MCU) ou quer um baseline ultra rápido.	poder computacional sobrando e quer a métrica mais estável a iluminação e a ROI; útil como validação cruzada.

Recomendação prática para o teste

1. Implementar as quatro e rodar a varredura completa (40 mm, ~25 pontos) sobre a mesma lâmina. Gerar as 4 curvas score × z sobrepostas;
2. Métrica primária: Tenengrad. É o melhor equilíbrio robustez × custo para lâminas histológicas com iluminação controlada;
3. Métrica de validação cruzada: FFT. Se Tenengrad e FFT concordarem no pico, é uma forte evidência experimental, rende um bom gráfico comparativo;
4. Métrica de controle: Laplaciano como baseline (literatura) e Brenner como referência de baixo custo;
5. Documentar o tempo médio por captura × métrica em microscopia, isso pesa na decisão final.